MyCodeSchool

# User Management în React — fără server

9 exerciții mici · doar ce ai învățat la inițiere · Rareș

**Pornire:** `npm install` (o dată), apoi `npm run dev` → `http://localhost:5173`
**Unde scrii:** doar în `src/users/UserList.tsx`.
**Cum arată:** deschide `mockup/index.html` în browser; HTML-ul îl copiezi de acolo (`class` devine `className`).

## 1. Tipul `User`

În `UserList.tsx`, deasupra funcției, descrie un utilizator.

```
type User = {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  age: number;
};
```

## 2. Lista de utilizatori

În funcție, o stare cu 5 utilizatori (ca la filme, ex. 7b).

```
const [users, setUsers] = useState<User[]>([
  { id: 1, firstName: "Bogdan", lastName: "Alexandrescu", email: "bogdan@gmail.com", age: 41 },
  { id: 2, firstName: "Radu", lastName: "Popescu", email: "radu@gmail.com", age: 17 },
  ...încă 3, inventați de tine
]);
```

**Indiciu:** sus în fișier, `import { useState } from "react";`

## 3. Câți sunt?

Un titlu cu numărul de utilizatori.

**Indiciu:** `<h1 className="page-title">Utilizatori ({users.length})</h1>`

**Pe ecran:** Utilizatori (5)

## 4. Tabelul

Copiază tabelul din mockup (fără coloana Identificator) și lasă un singur rând, în `map`.

```
{users.map((user) => (
  <tr key={user.id}>
    <td>{user.firstName} {user.lastName}<br />{user.email}</td>
    <td>{user.age} ani</td>
    <td><button>Editeaza</button> <button>Sterge</button></td>
  </tr>
))}
{users.length === 0 && <tr><td colSpan={3}>Niciun utilizator</td></tr>}
```

**Indiciu:** avatarul cu inițiale = `{user.firstName[0]}{user.lastName[0]}`.

**Pe ecran:** 5 rânduri.

## 5. Minor sau adult

Vârsta apare galben sub 18 ani și gri în rest.

```
<span className={user.age < 18 ? "badge badge-warning" : "badge badge-neutral"}>
  {user.age} ani
</span>
```

**Pe ecran:** doar Radu (17) e galben.

## 6. Șterge

Butonul „Sterge” scoate utilizatorul din listă.

```
function remove(id: number): void {
  setUsers(users.filter((user) => user.id !== id));
}
```

**Indiciu:** pe buton, `onClick={() => remove(user.id)}`.

**Gata când:** ștergi Radu → Utilizatori (4); ștergi pe toți → „Niciun utilizator”.

## 7. Adaugă

Deasupra tabelului: câmpurile Prenume, Nume, Email, Varsta și butonul „Salveaza”. Fiecare câmp are starea lui (input controlat, ca în Demo 5).

```
const [firstName, setFirstName] = useState<string>("");   // la fel: lastName, email
const [age, setAge] = useState<number>(0);
const [nextId, setNextId] = useState<number>(6);

<input value={firstName} onChange={(e) => setFirstName(e.target.value)} />
<input type="number" value={age} onChange={(e) => setAge(Number(e.target.value))} />
```

Funcția `save()` face un utilizator nou, îl pune la finalul listei și golește câmpurile:

```
const user: User = { id: nextId, firstName, lastName, email, age };
setNextId(nextId + 1);
setUsers([...users, user]);
setFirstName("");                        // la fel pentru celelalte
```

**De ce stare și pentru `nextId`:** un `let` simplu se resetează la fiecare redesenare (Demo 3).

**Gata când:** adaugi un utilizator → apare ultimul în tabel, câmpurile se golesc.

## 8. Editează

„Editeaza” pune datele utilizatorului în câmpuri; „Salveaza” îl modifică în loc să adauge unul nou.

```
const [editedId, setEditedId] = useState<number>(0);   // 0 = adaug, altfel = editez

function edit(user: User): void {
  setEditedId(user.id);
  setFirstName(user.firstName);          // la fel pentru celelalte
}
```

În `save()`: dacă `editedId` e `0`, adaugi ca la ex. 7; altfel înlocuiești:

```
setUsers(users.map((user) =>
  user.id === editedId ? { id: user.id, firstName, lastName, email, age } : user));
setEditedId(0);
```

**Gata când:** Radu → Editeaza → vârsta 18 → Salveaza: badge-ul lui devine gri.

## 9. Nu salva goluri

„Salveaza” e inactiv cât timp lipsește prenumele, numele sau vârsta.

```
const canSave: boolean = firstName !== "" && lastName !== "" && age > 0;

<button disabled={!canSave} onClick={save}>Salveaza</button>
```

**Gata când:** la pornire butonul e gri; după ce completezi câmpurile devine activ.

## Bonus

**Căutare** — o stare `search` și `const filtered: User[] = users.filter(...)`; tabelul face `map` pe `filtered`. *Indiciu:* `.toLowerCase().includes(text)`.

**Statistici** — vârsta medie, minori, adulți, fiecare o variabilă calculată în funcție. *Indiciu:* minori = `users.filter((u) => u.age < 18).length`; suma = `reduce` (Demo 10).

**Confirmare la ștergere** — `const [toDelete, setToDelete] = useState<User | null>(null)`; „Sterge” doar îl setează, iar fereastra din mockup apare cu `{toDelete !== null && (...)}`.